COMP47780-CLOUD COMPUTING-2025/26 AUTUMN

Name: Hemanathan Sasikala Karthikeyan

Student ID: 25201772

Github Link: https://github.com/Hemanathann/Hospital-Bed-capacity-analytics-aws-bi.git

1. APPLICATION OVERVIEW

Hospitals constantly juggle two things that do not always move together: how many beds and staff are available, and how many patients are actually arriving. If capacity is too low, patients are refused or left waiting. If capacity is too high, beds and staff are under-used and money is wasted.

For this coursework I chose **Project 1: Healthcare data warehouse management** and implemented it using AWS (S3, Glue, Athena, EC2) plus Power BI and Streamlit. In this project I built a cloud-based analytics solution that tracks this balance for four core hospital services - Emergency, ICU, General Medicine and Surgery- across 52 weeks.

The solution starts with data explored in a Jupyter notebook, then stored and processed on AWS (S3, Glue and Athena), feeds that data into Power BI for rich interactive dashboards, and finally exposes a similar view as a Streamlit web application deployed on an EC2 instance. The application allows hospital managers to:

- Monitor weekly demand vs. admitted patients and available beds.
- Quantify where bed shortages are building up.
- Compare patient satisfaction and staff morale across services.
- Spot services that are overstretched or where refusal rates are unusually high.

Overall, the system behaves like a small "capacity command centre" for a hospital, but implemented entirely with cloud and analytics tools that scale beyond this synthetic dataset.

2. OBJECTIVES

The project objectives and status are summarised below.

Objective	Status	Notes
Ingest and store hospital activity data in AWS S3	**Completed**	Raw CSV files uploaded to S3 in a structured folder layout.
Explore and engineer features in a Jupyter notebook	**Completed**	**hospital_beds_eda.ipynb** used for initial EDA, sanity checks and creation of derived metrics.
Process and aggregate data using a cloud compute engine (Glue cluster)	**Completed**	Glue job (Spark) cleans and prepares weekly service-level metrics.

Provide SQL access to the processed data using Athena	**Completed**	Athena table created over S3 processed files; final dataset exported as CSV.
Build an interactive dashboard in Power BI	**Completed**	Two-page report: “Capacity vs Demand” and “Service & Staff Experience”.
Re-create the key insights in a Python web app (Streamlit)	**Completed**	app.py mirrors the Power BI KPIs and charts with tabs and filters.
Deploy the analytics view on a public cloud endpoint (EC2)	**Completed**	Streamlit app deployed on Ubuntu EC2 instance and reachable via public URL.
Document the architecture, workflow and findings in a written report	**Completed**	This document.

All of the initial goals for the chosen project option have therefore been met.

3. PROBLEM DESCRIPTION AND DATASET

3.1 PROBLEM STATEMENT

Hospitals rarely suffer from a complete lack of beds; more often the problem is **mis-matched capacity**. Some services run at 100% utilisation with high refusal rates, while others have spare beds but not the right staff or equipment. Decisions such as opening extra beds, scheduling staff, or redirecting patients need evidence rather than intuition.

The problem I address is:

“Given a year of weekly demand, capacity and experience data for a hospital, identify where bed shortages occur, which services are overstretched, and how this affects patient satisfaction and staff morale.”

3.2 DATASET

The final dataset used for analysis is service_capacity_staff.csv. It is a synthetic but realistic dataset created and processed via the AWS pipeline. Each row represents one service–week combination, so there are 4 services × 52 weeks = 208 rows.

The key fields are:

- week – week number (1–52).
- service – one of: emergency, ICU, general_medicine, surgery.
- patients_request – number of patients requesting admission.
- patients_admitted – number actually admitted.
- patients_refused – number refused due to capacity.
- available_beds – beds staffed and available for that service and week.
- patient_satisfaction – average satisfaction score (0–100).
- staff_morale – average staff morale score (0–100).

During processing I derived the following additional measures:

- bed_shortage = max(patients_request − available_beds, 0)
- spare_capacity = max(available_beds − patients_request, 0)
- utilisation_rate = patients_admitted / available_beds × 100
- refusal_rate = patients_refused / patients_request × 100

When aggregated over all weeks, the numbers paint an interesting picture:

Service	BedShortage (patients)	Spare Bed Capacity (beds)	Utilisation %	Avg Satisfaction	Avg Morale	Refusal %
Emergency	5008	0	100.00	77.88	73.56	80.87
ICU	17	124	83.94	81.62	70.98	17.87
General Medicine	1866	72	97.00	81.23	73.10	45.39
Surgery	290	265	86.42	79.27	72.63	24.77
Overall	7181	461	92.70	80.00	72.57	56.64

These values are used throughout the dashboards and in the worked example.

4. METHODOLOGY AND IMPLEMENTATION

The overall architecture follows a simple but realistic cloud pattern:

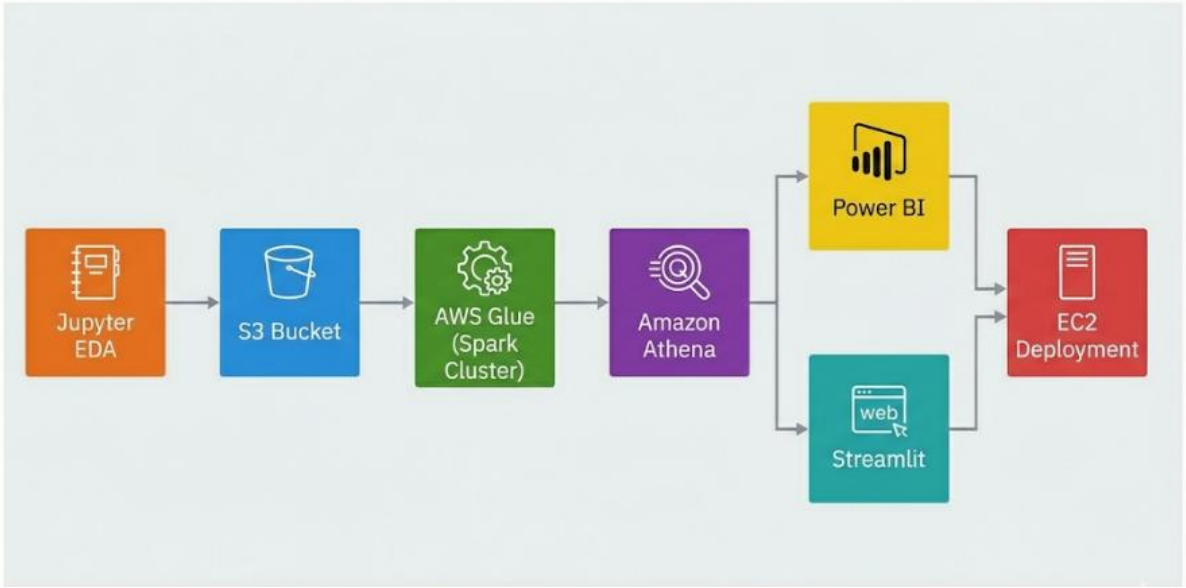


Figure 1: Overall cloud architecture – pipeline from Jupyter to EC2

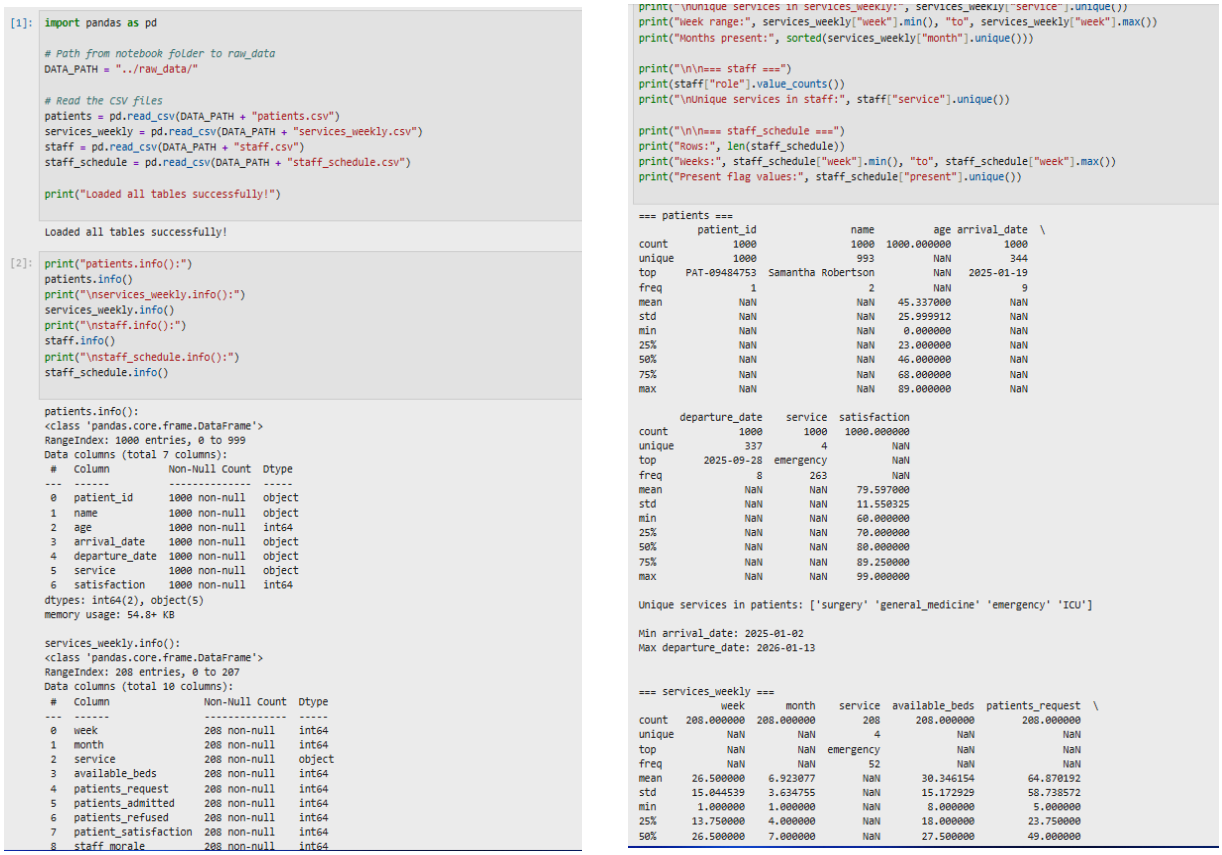
4.1 EXPLORATORY ANALYSIS IN JUPYTER

Before moving anything into the managed AWS services, I worked locally in a Jupyter notebook called ***hospital_beds_eda.ipynb***. This step helped me understand the data and fix issues before "locking in" the pipeline.

In the notebook I:

- Loaded the raw hospital CSV files using Pandas.
- Checked basic data quality: row counts, date/week range, missing values and obvious outliers.
- Created and validated the derived metrics (bed_shortage, spare_capacity, utilisation_rate, refusal_rate).
- Produced quick line plots by week and simple group by service to see if the numbers behaved sensibly. For example, I confirmed that Emergency consistently showed higher demand and shortages than the other services.

Once the logic looked correct and the distributions made sense, I exported the cleaned table as ***service_capacity_staff.csv***, which then became the single source of truth for the rest of the project.



```
[1]: import pandas as pd

# Path from notebook folder to raw_data
DATA_PATH = "../raw_data/"

# Read the CSV files
patients = pd.read_csv(DATA_PATH + "patients.csv")
services_weekly = pd.read_csv(DATA_PATH + "services_weekly.csv")
staff = pd.read_csv(DATA_PATH + "staff.csv")
staff_schedule = pd.read_csv(DATA_PATH + "staff_schedule.csv")

print("Loaded all tables successfully!")

Loaded all tables successfully!

[2]: print("patients.info():")
patients.info()
print("\nservices_weekly.info():")
services_weekly.info()
print("\nstaff.info():")
staff.info()
print("\nstaff_schedule.info():")
staff_schedule.info()

patients.info():
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1000 entries, 0 to 999
Data columns (total 7 columns):
 #   Column          Non-Null Count  Dtype
---  ------          --------------  -----
 0   patient_id      1000 non-null   object
 1   name            1000 non-null   object
 2   age             1000 non-null   int64
 3   arrival_date    1000 non-null   object
 4   departure_date  1000 non-null   object
 5   service         1000 non-null   object
 6   satisfaction    1000 non-null   int64
dtypes: int64(2), object(5)
memory usage: 54.8+ KB

services_weekly.info():
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 208 entries, 0 to 207
Data columns (total 10 columns):
 #   Column               Non-Null Count  Dtype
---  ------               --------------  -----
 0   week                 208 non-null    int64
 1   month                208 non-null    int64
 2   service              208 non-null    object
 3   available_beds       208 non-null    int64
 4   patients_request     208 non-null    int64
 5   patients_admitted    208 non-null    int64
 6   patients_refused     208 non-null    int64
 7   patient_satisfaction 208 non-null    int64
 8   staff_morale         208 non-null    int64
```

```
print("\nUnique services in services_weekly:", services_weekly["service"].unique())
print("Week range:", services_weekly["week"].min(), "to", services_weekly["week"].max())
print("Months present:", sorted(services_weekly["month"].unique()))

print("\n\n=== staff ===")
print(staff["role"].value_counts())
print("\nUnique services in staff:", staff["service"].unique())

print("\n\n=== staff_schedule ===")
print("Rows:", len(staff_schedule))
print("Weeks:", staff_schedule["week"].min(), "to", staff_schedule["week"].max())
print("Present flag values:", staff_schedule["present"].unique())

=== patients ===
        patient_id               name          age arrival_date  \
count         1000               1000  1000.000000         1000
unique        1000                993          NaN          344
top     PAT-09484753  Samantha Robertson          NaN   2025-01-19
freq             1                  2          NaN            9
mean           NaN                NaN    45.337000          NaN
std            NaN                NaN    25.999912          NaN
min            NaN                NaN     0.000000          NaN
25%            NaN                NaN    23.000000          NaN
50%            NaN                NaN    46.000000          NaN
75%            NaN                NaN    68.000000          NaN
max            NaN                NaN    89.000000          NaN

      departure_date    service  satisfaction
count           1000       1000   1000.000000
unique           337          4           NaN
top       2025-09-28  emergency           NaN
freq               8        263           NaN
mean             NaN        NaN     79.597000
std              NaN        NaN     11.550325
min              NaN        NaN     60.000000
25%              NaN        NaN     70.000000
50%              NaN        NaN     80.000000
75%              NaN        NaN     89.250000
max              NaN        NaN     99.000000

Unique services in patients: ['surgery' 'general_medicine' 'emergency' 'ICU']

Min arrival_date: 2025-01-02
Max departure_date: 2026-01-13


=== services_weekly ===
              week       month    service  available_beds  patients_request  \
count   208.000000  208.000000        208      208.000000        208.000000
unique         NaN         NaN          4             NaN               NaN
top            NaN         NaN  emergency             NaN               NaN
freq           NaN         NaN         52             NaN               NaN
mean     26.500000    6.923077        NaN       30.346154         64.870192
std      15.044539    3.634755        NaN       15.172929         58.738572
min       1.000000    1.000000        NaN        8.000000          5.000000
25%      13.750000    4.000000        NaN       18.000000         23.750000
50%      26.500000    7.000000        NaN       27.500000         49.000000
```

Figure 2: Jupyter notebook (hospital_beds_eda.ipynb) used for initial EDA

4.2 DATA STORAGE ON S3

I then created an S3 bucket and uploaded the raw and processed CSV files. The files were organised into folders such as:

- **raw/** – original input CSVs.
- **processed/** – cleaned and aggregated files produced by Glue, including service_capacity_staff.csv.

Using S3 gave me durable storage, versioning, and made it easy to plug the same data into both Athena and external tools.

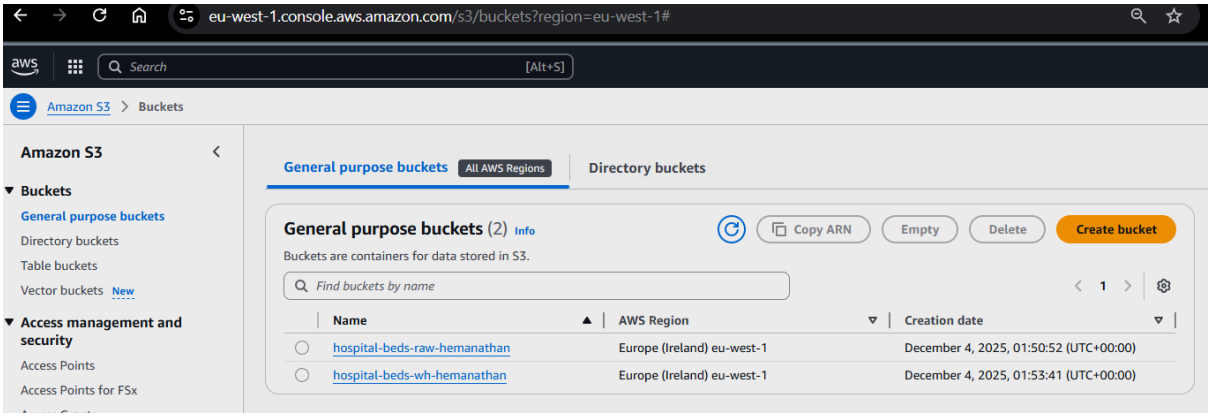


Figure 3: Hospital dataset stored in S3

4.3 DATA PREPARATION WITH AWS GLUE CRAWLERS & DATA CATALOG

Instead of writing a full Spark ETL job, I used AWS Glue Crawlers and the Glue Data Catalog to prepare the data for Athena and the dashboards. Glue acts as the metadata layer for everything stored in S3, so once the crawlers are configured, any tool that speaks to the Data Catalog can query the same tables.

I set up two crawlers:

1. hospital_raw_crawler - points at the raw folder in the S3 bucket and automatically creates tables such as:

- raw_patients_csv
- raw_services_weekly_csv
- raw_staff_csv
- raw_staff_schedule_csv

2. hospital_wh_crawler - points at the warehouse/processed area and registers the final, analytics-ready view:

- vw_service_capacity_staff

Each time a crawler runs, Glue scans the CSV files in S3, infers the schema (column names and types), and updates the Glue Data Catalog. This means Athena can immediately query the same tables without me having to define schemas manually.

The final table/view vw_service_capacity_staff is the one that contains the weekly, per-service metrics (including bed_shortage, spare_capacity, utilisation_rate and refusal_rate). This table is what I use in Athena for validation and what I export as service_capacity_staff.csv for Power BI and the Streamlit app.

Although the dataset is small, using Glue crawlers and the catalog is useful because the same setup would scale to much larger hospital datasets with minimal changes.

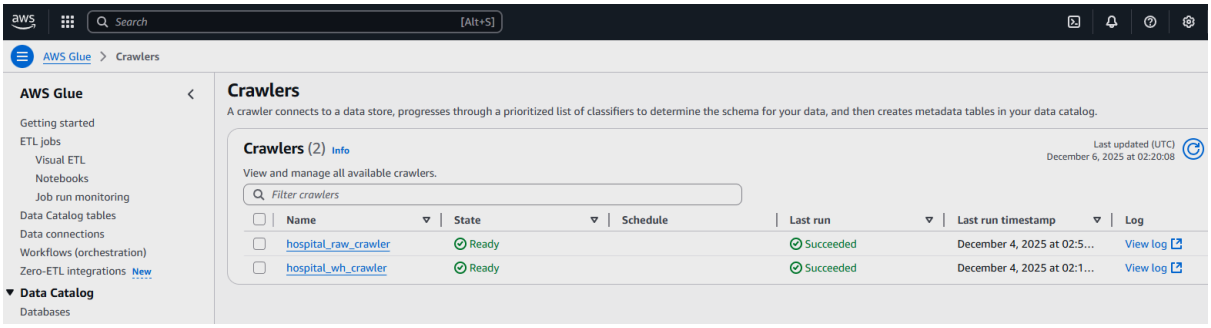


Figure 3: AWS Glue crawlers for raw and warehouse data

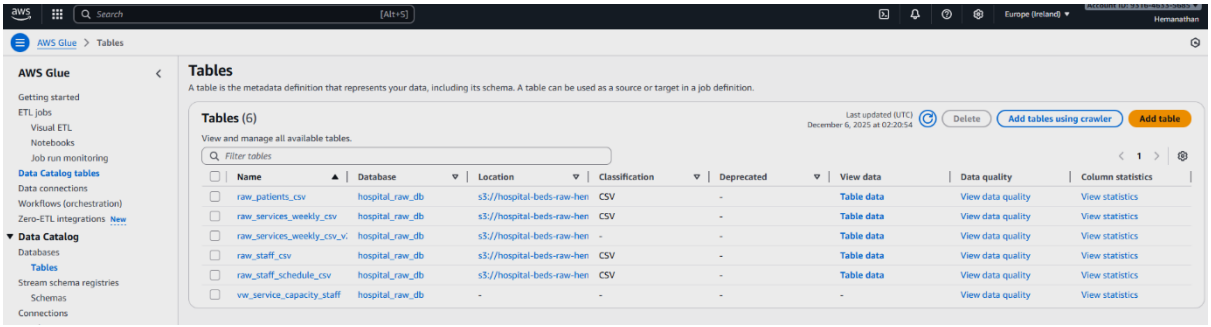


Figure 4: Glue Data Catalog tables feeding Athena

4.4 QUERYING AND COMBINING DATA USING ATHENA

Once the raw and derived tables were available in the Glue Data Catalog, I used Amazon Athena as a serverless SQL layer on top of S3. The goal here was to pull together the weekly service data and the staff-schedule data, and to calculate the main KPIs in one place before exporting the final dataset.

Using the tables *raw_services_weekly_csv_v2* and *raw_staff_schedule_csv* in the *hospital_raw_db* database, I created a view called ***vw_service_capacity_staff***. This view aggregates staff presence by service and week, joins it to the weekly service metrics, and computes the two key rates used throughout the project: *utilisation_rate* and *refusal_rate.*

A simplified version of the SQL is shown below:

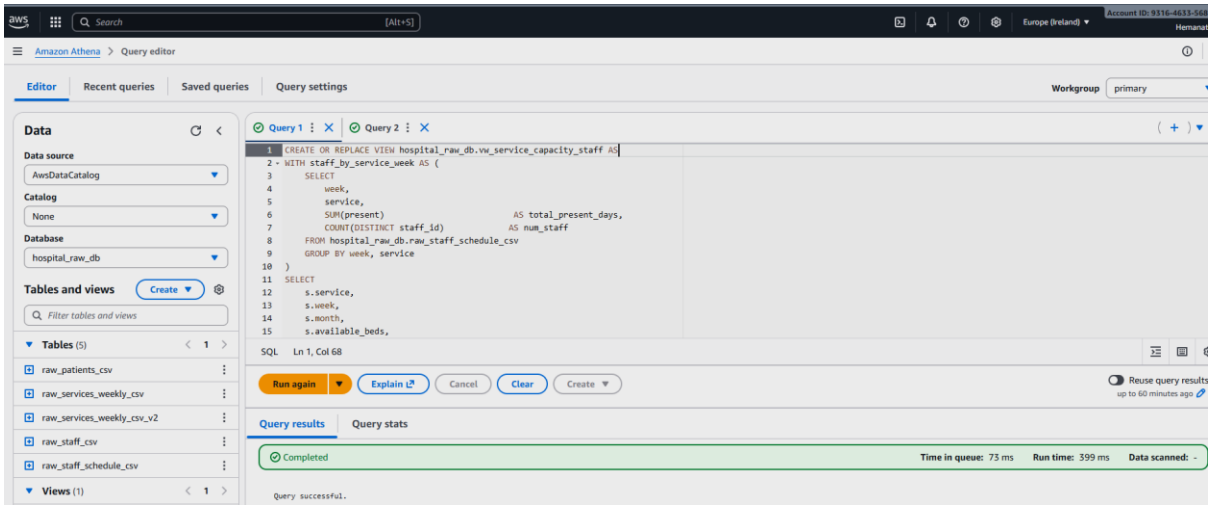


Figure 5: Athena query editor – creating the *vw_service_capacity_staff* view used by Power BI and Streamlit.

After creating the view, I ran a simple ***SELECT * FROM hospital_raw_db.vw_service_capacity_staff LIMIT 10***; to sanity-check the numbers against the Jupyter EDA. Once I was happy that the figures matched, I used Athena's "Download results" option to export the view as service_capacity_staff.csv. This CSV is the single, analytics-ready dataset that feeds both the Power BI report and the Streamlit dashboard described in the next sections.

4.5 POWER BI REPORTING

I then imported ***service_capacity_staff.csv*** into Power BI Desktop and built a two-page report.

PAGE 1: CAPACITY VS DEMAND OVERVIEW

This page is designed for a quick "year on a page" view:

- A weekly area/line chart shows patient requests, patients admitted and available beds across 52 weeks.
- A second chart titled "Distribution of Demand, Admissions & Capacity by Week" uses bars for requests and lines for admissions and beds to emphasise the gaps.
- A set of KPI tiles summarises:
  - Total bed shortage: **7,181** patients.
  - Average bed utilisation: **92.70%.**
  - Average patient satisfaction: **80.00**.
  - Average refusal rate: **56.64%.**
  - Spare bed capacity: **461 beds**.

A service slicer in the centre lets the user click Emergency, ICU, General Medicine or Surgery and see all numbers recomputed for that service only.

When the slicer is set to Emergency, for example, the page shows a bed shortage of **5,008 patients**, utilisation fixed at **100%**, high refusal **(80.87%)** and lower satisfaction **(77.88)**.

For ICU, the bed shortage drops to just **17 patients**, with better satisfaction **(81.62)** and a much lower refusal rate **(17.87%).**

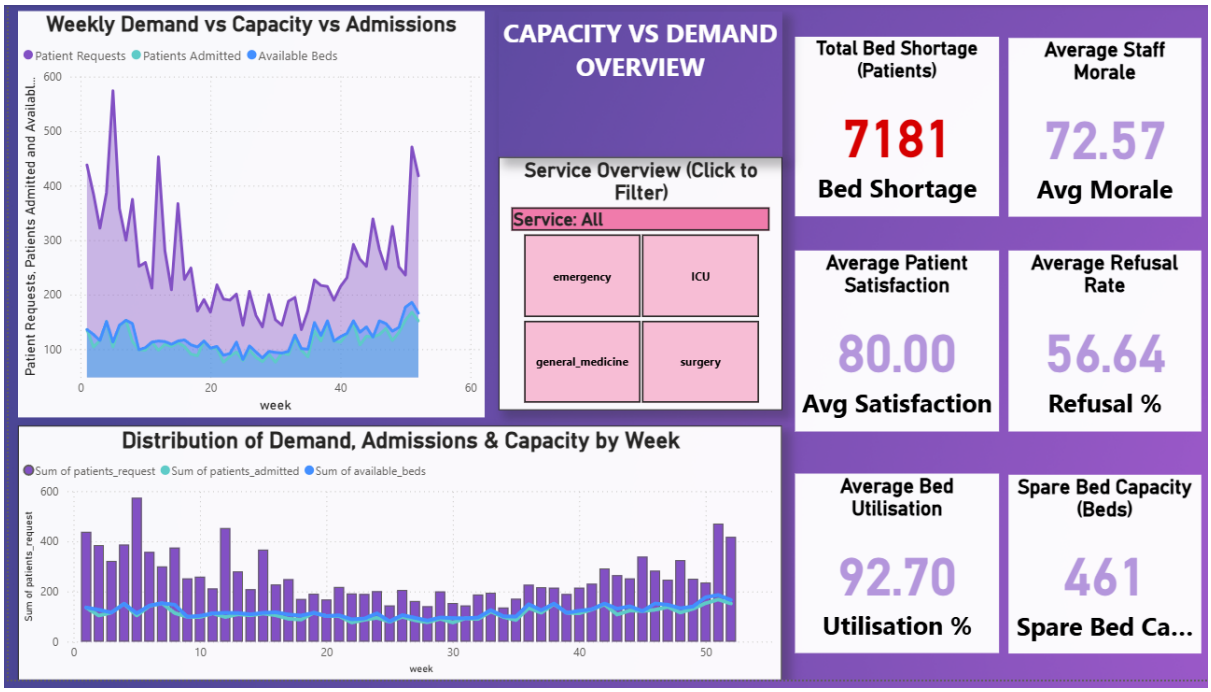


Figure 6: Power BI – Capacity vs Demand overview (All services)

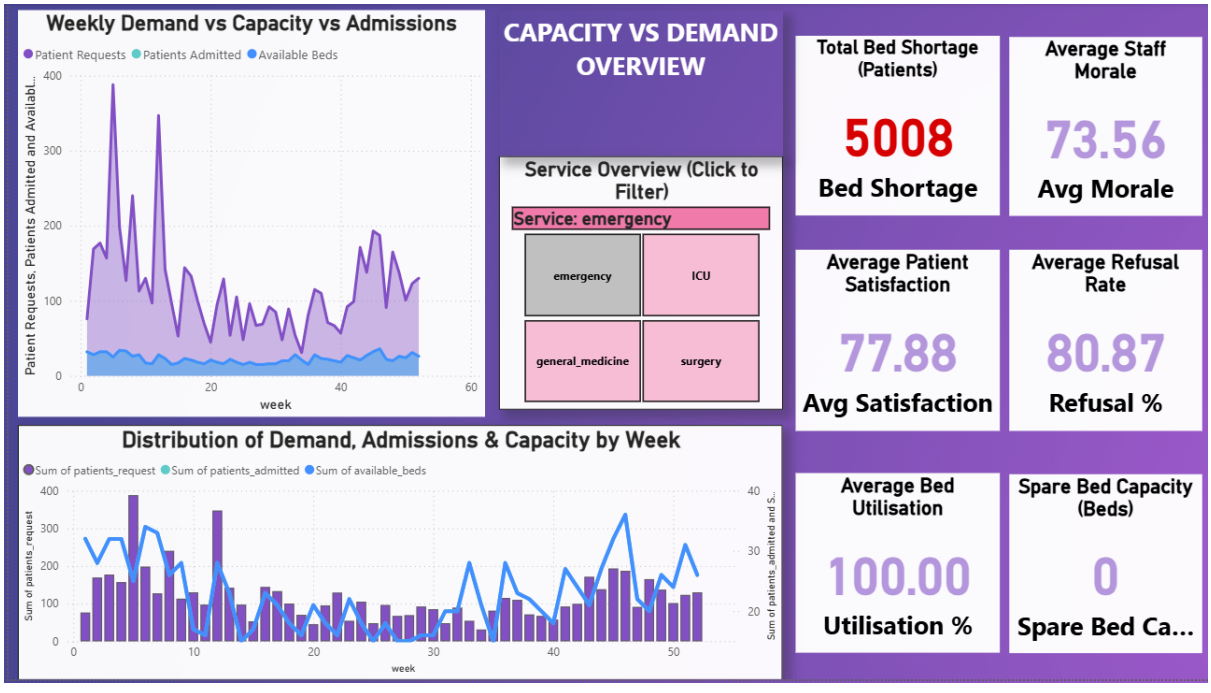


Figure 7: Power BI – Capacity vs Demand overview filtered on Emergency

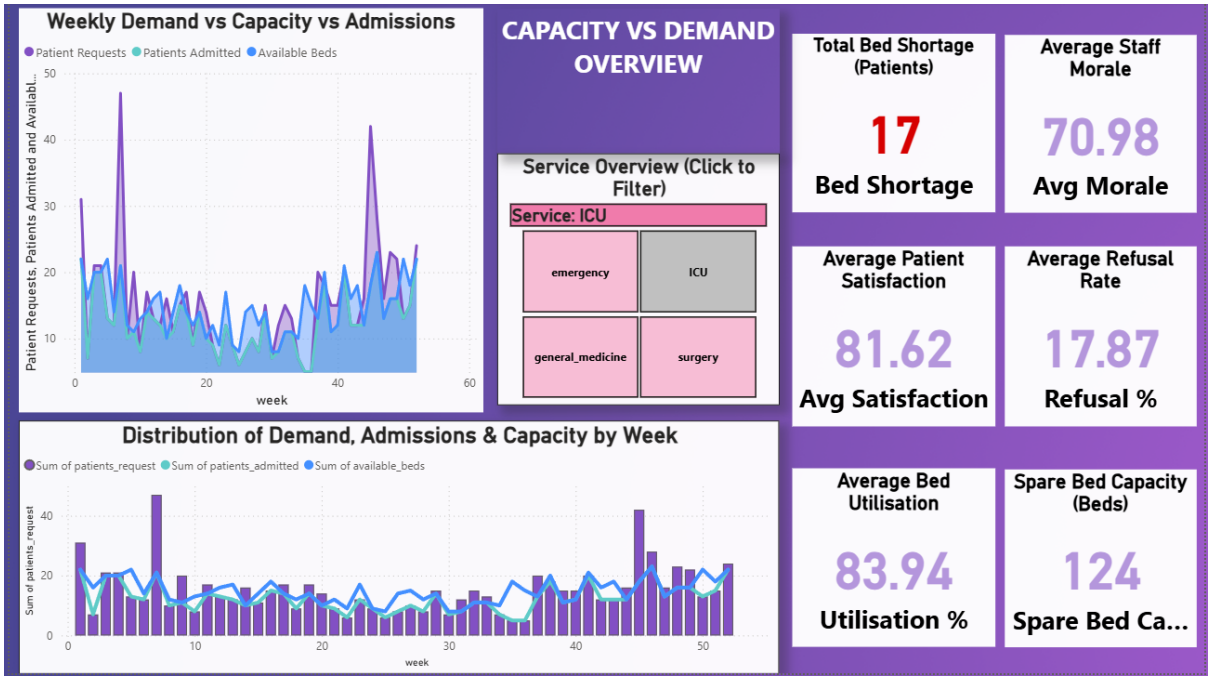


Figure 8: Power BI – Capacity vs Demand overview filtered on ICU

PAGE 2: SERVICE & STAFF EXPERIENCE

The second page focuses more on experience and pressure by service:

- A bar chart shows Average Refusal Rate by Service (%). Emergency clearly stands out at around 81%, while ICU sits near 18%.
- A scatter/bubble chart titled "Staff Morale vs Patient Satisfaction (Bubble size = Bed Shortage)" plots each service:
  - X-axis: average staff morale.
  - Y-axis: average patient satisfaction.
  - Bubble size: accumulated bed shortage.
  - Emergency appears as the largest bubble with relatively lower satisfaction, indicating a stressed service.
- A detailed table summarises utilisation %, refusal %, satisfaction, morale, bed shortage and spare capacity by service. This matches the numbers in the table in Section 3.2.
- KPIs highlight:
  - Number of overstretched services (utilisation ≥ 95% and bed shortage > 0): 2.
  - Number of services hitting satisfaction target (score ≥ 80): 2.
- A short "Key Insights" text card summarises what the numbers reveal.

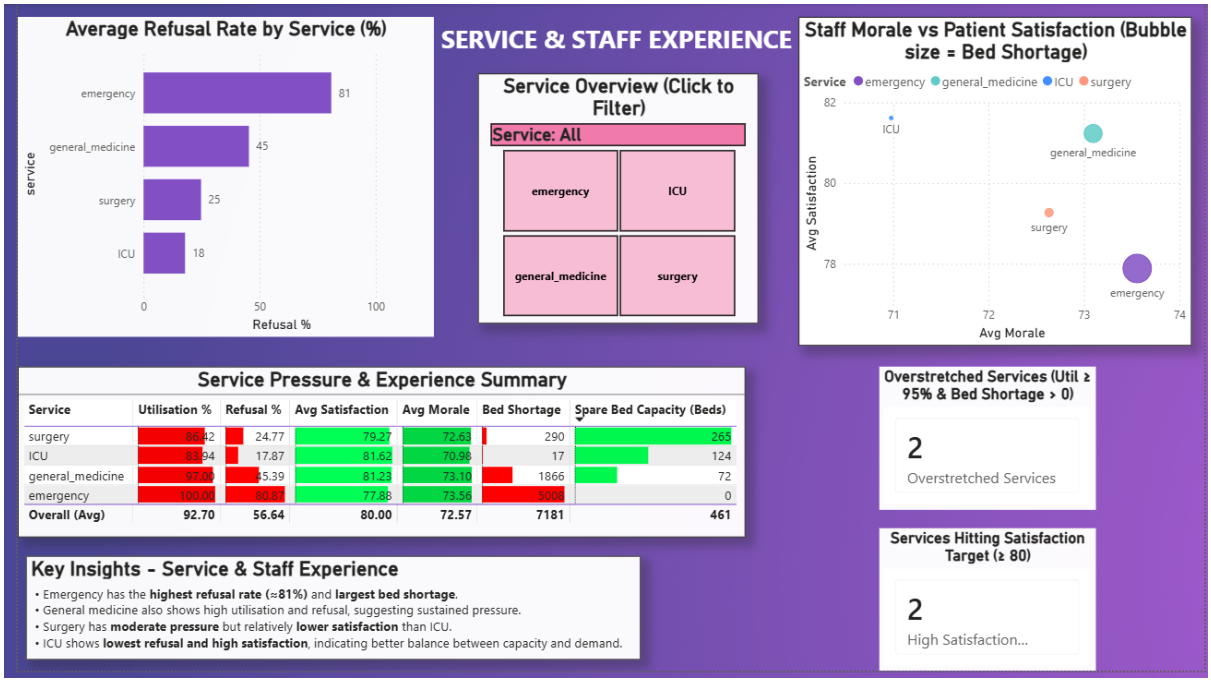


Figure 9: Power BI – Service & Staff Experience page

4.6 STREAMLIT APP (PYTHON)

Power BI is excellent for desktop analysis, but I also wanted a lightweight, code-based dashboard that could be hosted in the cloud. For that, I built a Streamlit application (***app.py***) using Pandas and Altair.

Key points:

- The app reads the same *service_capacity_staff.csv* file into a Pandas dataframe.
- It creates a sidebar with slicer-like filters:
  - A list of services with a "Select all services" checkbox, implemented as independent checkboxes for ICU, Emergency, General Medicine and Surgery.
  - A week range slider (1–52).
- The main view uses two tabs, mirroring Power BI:
  - "**Capacity vs Demand Overview**" – KPIs plus line and layered bar/line charts.
  - "**Service & Staff Experience**" – refusal bar chart, bubble chart, summary table and text insights.
- The logic for KPIs and charts is completely transparent in Python, and the code is short enough for another developer to pick up quickly.

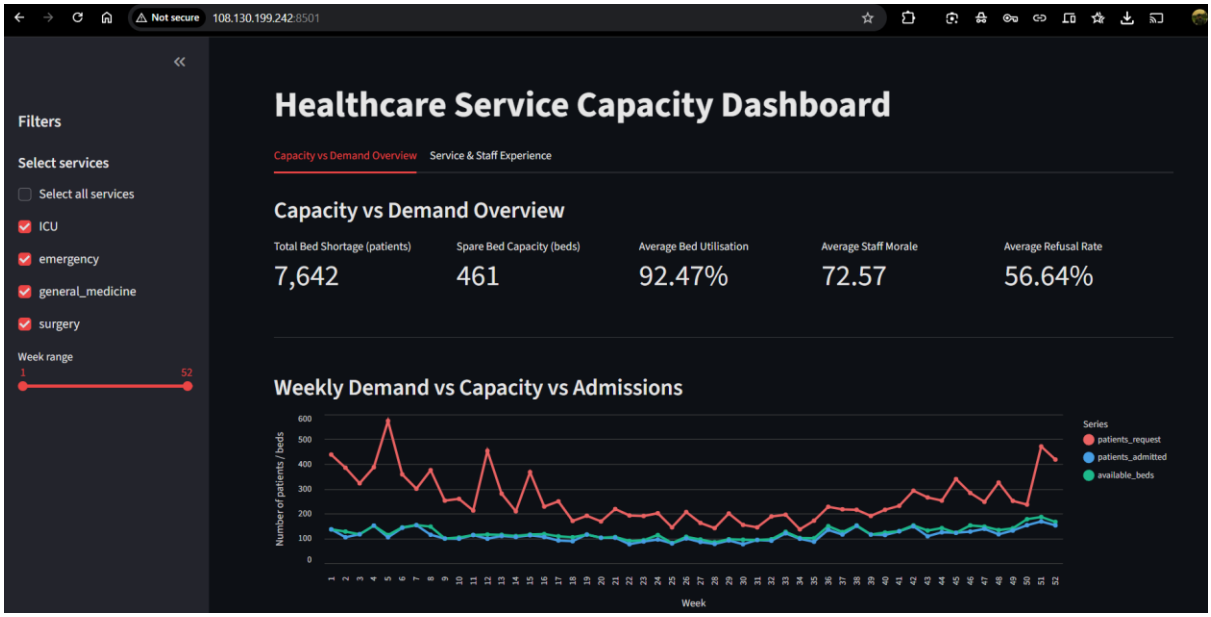


Figure 10: Streamlit app – Capacity vs Demand tab

Figure 11: Streamlit app – Service & Staff Experience tab

4.7 DEPLOYMENT ON EC2

To make the analytics available via a public URL, I deployed the Streamlit app on an Ubuntu EC2 instance:

1. Created a t3.micro instance using Ubuntu 24.04.
2. Opened inbound security group rules for ports 22 (SSH) and 8501 (Streamlit default).
3. Connected via SSH using the key pair ***heman-streamlit-key.pem***.

4. Created a folder ***~/streamlit_app*** and uploaded ***app.py, requirements.txt*** and ***service_capacity_staff.csv*** using ***scp.***
5. Set up a Python virtual environment, installed dependencies from ***requirements.txt***, and started the app with:
   - ***streamlit run app.py --server.port 8501 --server.address 0.0.0.0***
6. Accessed the dashboard from my browser using:
   - ***http://108.130.199.242:8501/***
7. For long-running availability I can also run it under ***nohup*** or a process manager, but for this coursework a manually started session is sufficient.

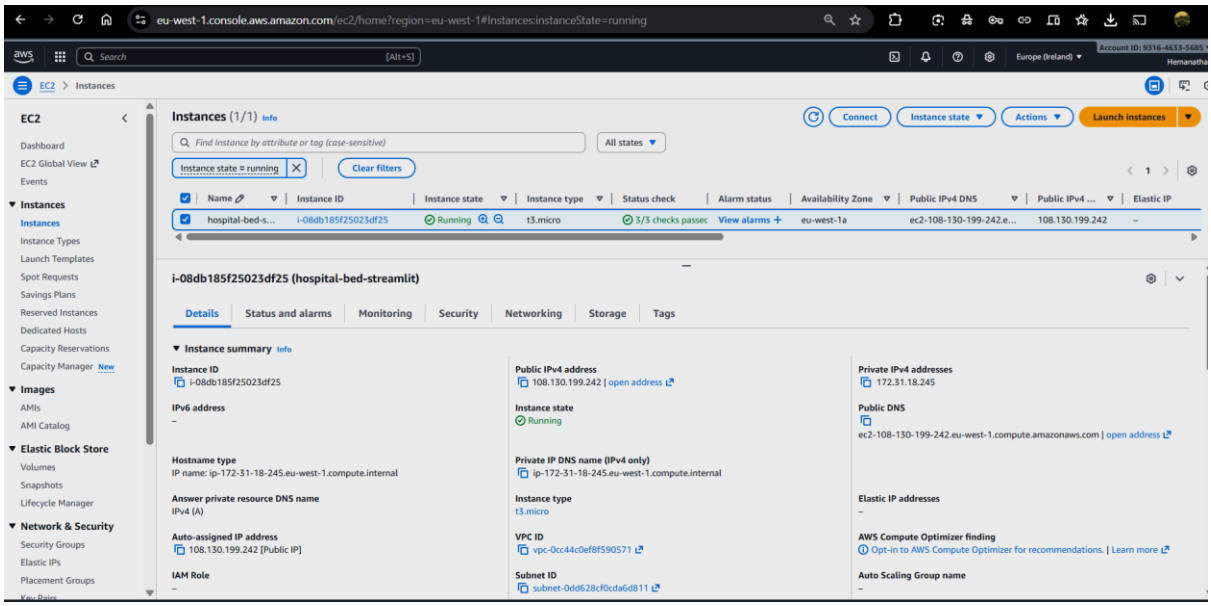


Figure 12: EC2 instance hosting the Streamlit app

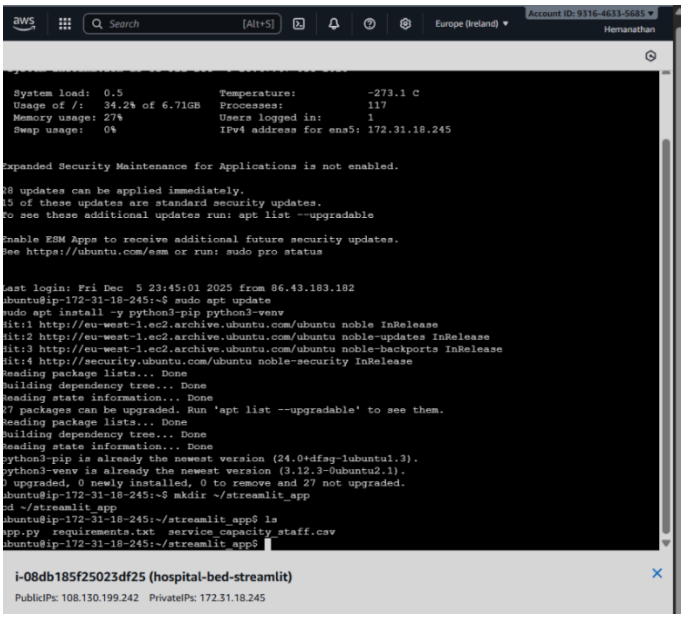


Figure 13: SSH session running the Streamlit app

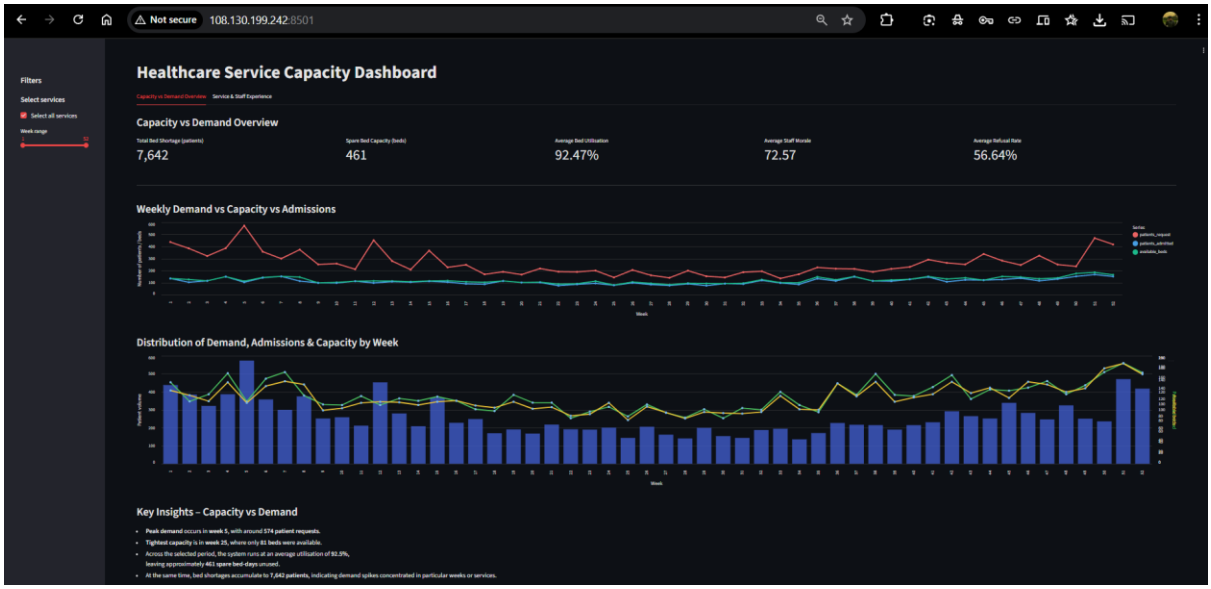


Figure 14: Public URL view of the Streamlit dashboard

5. SUITABILITY OF THE TOOLS

I chose the tools deliberately to match the nature of the problem.

- **Jupyter Notebook** is ideal for quick exploratory work. It let me experiment with metrics, check the data behaved sensibly and iterate on formulas without touching the cloud pipeline yet. Once I was happy, I pushed that logic into Glue and Streamlit.
- **Amazon S3** is a natural fit for storing raw and processed healthcare data because it is cheap, durable and supports virtually unlimited scale. The same data can be reused by Glue, Athena and any analytics tool that can read from S3.
- **AWS Glue (Spark cluster)** allowed me to implement all data preparation steps in one place. Even though the current dataset is small, the Spark engine means the same code could run on large hospital datasets covering multiple years or hospitals.
- **Amazon Athena** made it easy to run ad-hoc SQL without managing any database servers. For dataset validation and exporting the final table it was lighter and simpler than provisioning an RDS instance.
- **Power BI** is well suited to the "business user" perspective. It offers rich visual controls, interactive slicers, and a familiar layout for dashboards. It is especially good for quickly iterating through chart designs and tweaking labels and colours.
- **Streamlit** is a good fit for building a simple web UI on top of the same data using Python. It is lightweight, has a small learning curve, and lets me keep the code and the app in one place.
- **EC2** provides full control over the runtime environment for deployment. An alternative could have been a managed service like App Runner or Fargate, but for this coursework EC2 gave me better visibility of each step (SSH, virtualenv, ports, etc.).

The main limitation of this stack is that it is not fully serverless end-to-end: the Streamlit app on EC2 needs manual start-up and monitoring. For a production hospital deployment I would also want role-based access control, proper HTTPS, and automated scaling.

6. FEATURES OF THE SOFTWARE

From a user's point of view, the solution offers the following features:

UNIFIED CAPACITY VIEW

- All services and weeks in a single dashboard.
- Instant KPIs summarising total bed shortage, spare capacity, utilisation, satisfaction and refusal.

INTERACTIVE SERVICE FILTERING

- One click to switch between All, Emergency, ICU, General Medicine and Surgery.
- As soon as a service is selected, all charts and KPIs recalculate for that service only.

TIME-SERIES INSIGHTS

- Weekly curves allow users to spot spikes in demand or drops in capacity.
- Distribution charts highlight weeks where demand clearly exceeded beds.

EXPERIENCE AND PRESSURE ANALYTICS

- Comparison of refusal rates by service.
- Bubble chart showing how satisfaction and morale change when services are under pressure.
- Table that combines operational and experience metrics side by side.

DUAL FRONT-END

- Power BI report for rich analytical exploration.
- Streamlit web app for lightweight web access via a simple URL.

All of these features are driven by the same underlying dataset and metrics, so the story remains consistent whichever interface is used.

7. WORKED EXAMPLE

To illustrate how this system could actually support decisions, consider the following scenario.

7.1 SITUATION

A hospital operations manager wants to know why patients keep complaining about long waits in Emergency, and whether resources could be shifted from other services without causing new problems elsewhere.

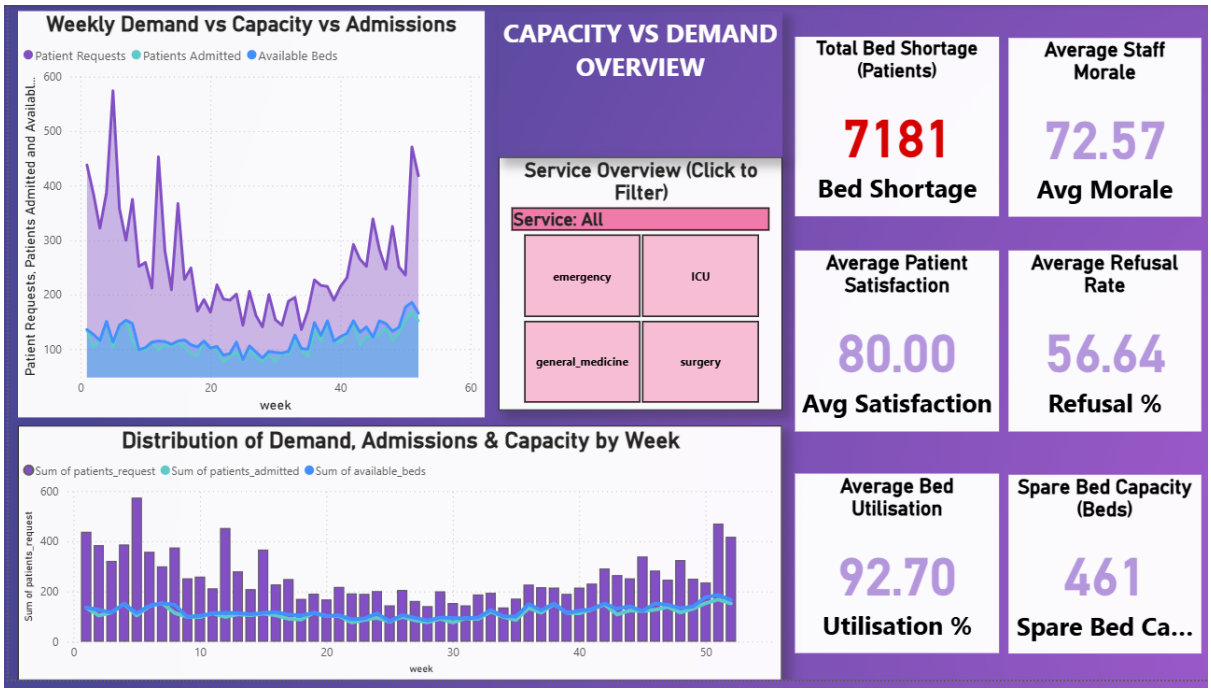
7.2 STEP 1 - VIEW OVERALL CAPACITY

The manager first opens the Capacity vs Demand Overview with all services selected.

The top KPIs show:

- Total bed shortage: 7,181 patients.
- Spare bed capacity: 461 beds.
- Average utilisation: 92.70%.

The immediate insight is that, overall, the hospital is running very "hot": little spare capacity and a lot of unmet demand.



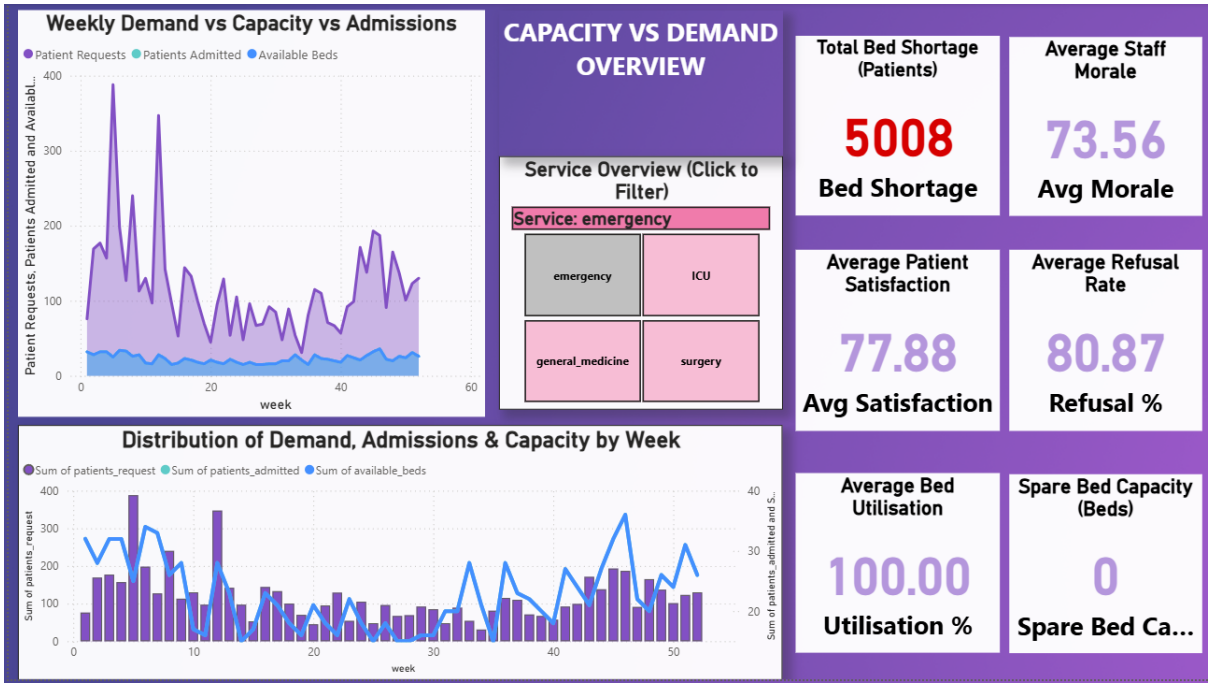
7.3 STEP 2 – FOCUS ON EMERGENCY

Next, the manager uses the service slicer and clicks Emergency.

The KPIs now change to:

- Bed shortage: 5,008 patients (the majority of the overall shortage).
- Utilisation: 100.00% almost every week.
- Refusal rate: 80.87%, far above any reasonable target.
- Average satisfaction: 77.88, noticeably lower than other services.

On the charts, the purple "*patient requests*" line sits consistently above the "*available beds*" line, confirming that Emergency simply does not have enough capacity for the demand hitting it.

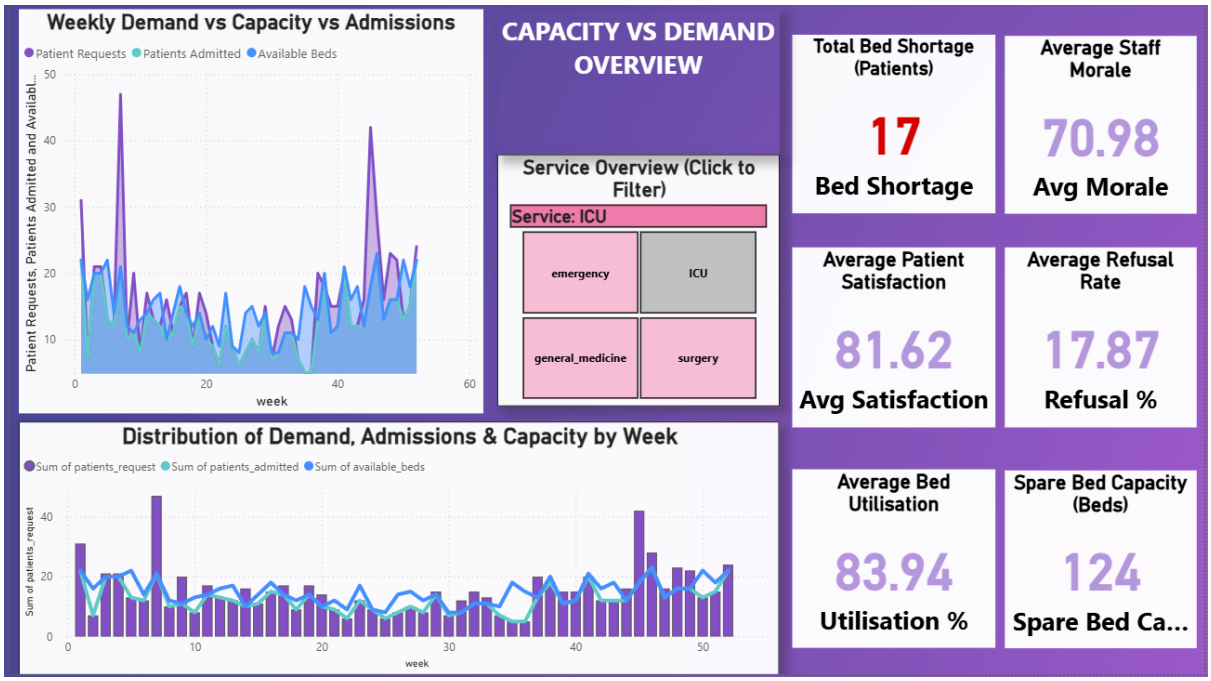


7.4 STEP 3 – COMPARE WITH ICU

The manager then selects ICU in the slicer.

- For ICU, bed shortage is only 17 patients over the entire year and refusal is 17.87%. Satisfaction at 81.62 is higher than in Emergency, and the service also has 124 spare beds across the year.

The weekly charts show periodic spare capacity, not continuous overload. This suggests ICU is better balanced and might be able to lend capacity in certain weeks.



7.5 STEP 4 – CHECK STAFF EXPERIENCE

Switching to the Service & Staff Experience page, the manager observes:

- Emergency has the highest refusal rate and largest bed shortage bubble, indicating serious pressure.
- ICU has much lower refusal and slightly higher satisfaction.
- The summary table confirms that Emergency and General Medicine are the two services classed as overstretched (utilisation ≥ 95% with shortages).

The combined picture suggests that:

- Emergency needs either more beds, more staff, or a redesign of patient flow.
- There is limited but non-zero room to rebalance some cases towards ICU or Surgery in selected weeks.

This is exactly the kind of reasoning the dashboards are designed to support.

8. CONCLUSION

This project demonstrates a complete, end-to-end cloud analytics pipeline for hospital capacity management:

- Data is explored in Jupyter, then stored and processed entirely on AWS using S3, Glue and Athena.
- The resulting dataset powers both a rich Power BI report and a lightweight Streamlit dashboard.
- The Streamlit app is deployed on an EC2 instance, making the visualisations available via a simple public URL.

From an analytical perspective, the system reveals that:

- The hospital accumulates 7,181 bed-shortage cases in a year, despite having 461 bed-days of spare capacity elsewhere.
- Emergency is the most pressured service, with very high refusal and shortage, while ICU and Surgery are relatively better balanced.
- Patient satisfaction and staff morale tend to drop when services run at or near full utilisation for long periods.

If I had more time, the next steps would be:

- Reading the dashboard data directly from Athena instead of a static CSV, enabling true near-real-time reporting.
- Adding simple forecasting models (for example, predicting next month's bed requirements) using Python.
- Implementing authentication and HTTPS for the EC2 deployment so that it could be safely used within a real hospital network.

Even in its current form, the project shows how a relatively small dataset, when combined with the right cloud and BI tools, can provide a clear and actionable view of healthcare service capacity.

Working through the full pipeline from raw CSVs in S3 to a live dashboard on EC2 helped me understand how the different AWS services fit together in a realistic healthcare setting.

9. REFERENCES

1. Amazon Web Services – Amazon S3 documentation.
2. Amazon Web Services – AWS Glue documentation.
3. Amazon Web Services – Amazon Athena documentation.
4. Amazon Web Services – Amazon EC2 documentation.
5. Microsoft – Power BI Desktop documentation.
6. Streamlit – Streamlit Python library documentation.